

EE332 数字系统设计

Lab 4–2&3: 序列信号发生与检测

姓 名: 何衍宁
学 号: 12311512

2026 年 3 月 25 日

目 录

1 lab4.2 序列信号发生器	2
1.1 实验目的	2
1.2 VHDL 代码和 testbench 代码	2
1.3 结果推导	4
1.4 结果	5
1.4.1 behavior simulation	5
1.4.2 post synthesis timing simulation	6
1.4.3 post implementation timing simulation	7
1.5 分析	7
1.6 结论	7
2 lab4.3 序列信号检测器	8
2.1 实验目的	8
2.2 VHDL 代码和 testbench 代码	8
2.3 结果推导	11
2.4 结果	12
2.4.1 behavior simulation	12
2.4.2 post synthesis timing simulation	13
2.4.3 post implementation timing simulation	14
2.5 分析	14
2.6 结论	14

1 lab4.2 序列信号发生器

1.1 实验目的

本次实验旨在通过 VHDL 硬件描述语言设计一个能够循环输出特定序列“1101110”的 Moore 型有限状态机 (FSM)，并编写对应的 Testbench 施加时钟与复位激励以验证其前端逻辑功能；同时，通过依次执行并对比行为级仿真、综合后时序仿真以及实现后时序仿真的波形结果，深入观察并理解实际数字系统底层由于逻辑门延迟和物理布线延迟所造成的时序偏移现象，从而全面掌握从代码编写、功能验证到后端物理时序分析的完整 EDA 硬件开发流程。

1.2 VHDL 代码和 testbench 代码

design 采用代码如下

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity signal_gen is
5      Port (
6          clk      : in  STD_LOGIC;
7          reset     : in  STD_LOGIC;
8          dataout   : out STD_LOGIC
9      );
10 end signal_gen;
11
12 architecture Behavioral of signal_gen is
13 -- 定义状态信号，使用与状态转移图一致的3位二进制编码
14     signal current_state : STD_LOGIC_VECTOR(2 downto 0);
15 begin
16     -- 状态转移同步进程
17     process(clk, reset)
18     begin
19         if reset = '1' then
20             current_state <= "000"; -- 高电平复位，初始化为000
21         elsif rising_edge(clk) then
22             case current_state is
23                 when "000" => current_state <= "001";
24                 when "001" => current_state <= "010";
25                 when "010" => current_state <= "011";
26                 when "011" => current_state <= "100";
27                 when "100" => current_state <= "101";
28                 when "101" => current_state <= "110";
29                 when "110" => current_state <= "000";
30                 when "111" => current_state <= "000"; -- 未使用状态的处理
31                 when others => current_state <= "000";
32             end case;
33         end if;
34     end process;
35
36     -- 输出组合逻辑进程 (Moore机：输出仅与当前状态有关)
37     process(current_state)
38     begin
39         case current_state is
40             when "000" => dataout <= '1';
41             when "001" => dataout <= '1';
42             when "010" => dataout <= '0';
```

```

43         when "011" => dataout <= '1';
44         when "100" => dataout <= '1';
45         when "101" => dataout <= '1';
46         when "110" => dataout <= '0';
47         when "111" => dataout <= 'X'; -- 对应图中的 /X
48         when others => dataout <= '0';
49     end case;
50 end process;
51 end Behavioral;

```

Testbench 通过独立进程生成了周期为 10ns 的时钟 (CLK) 及相关激励。代码在时序安排上刻意错开了 RESET 信号的下降沿与 CLK 的上升沿, 以此消除时序竞争风险。此外, 所施加的激励组合实现了对所有潜在状态的完全覆盖, 保障了仿真的全面性。仿真 testbench 的代码如下:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity testbench is
5  -- Testbench 没有端口
6  end testbench;
7
8  architecture tb of testbench is
9
10     -- 组件声明
11     component signal_gen
12         Port (
13             clk      : in  STD_LOGIC;
14             reset    : in  STD_LOGIC;
15             dataout  : out STD_LOGIC
16         );
17     end component;
18
19     -- 内部信号定义
20     signal clk      : STD_LOGIC := '0';
21     signal reset    : STD_LOGIC := '0';
22     signal dataout  : STD_LOGIC;
23
24     -- 定义时钟周期 (例如 10 ns -> 100 MHz)
25     constant clk_period : time := 10 ns;
26
27 begin
28
29     -- 实例化待测设计 (DUT)
30     uut: signal_gen Port map (
31         clk      => clk,
32         reset    => reset,
33         dataout  => dataout
34     );
35
36     -- 时钟生成进程
37     clk_process : process
38     begin
39         clk <= '0';
40         wait for clk_period/2;
41         clk <= '1';
42         wait for clk_period/2;
43     end process;
44

```

```

45     -- 激励施加进程
46     stim_proc: process
47     begin
48         -- 1. 初始化，施加复位信号
49         reset <= '1';
50         wait for 20 ns;
51
52         -- 2. 释放复位信号，开始正常工作
53         reset <= '0';
54
55         -- 3. 运行足够长的时间以观察多个完整的 "1101110" 序列循环
56         -- 一个完整序列需要 7 个时钟周期 (70 ns)
57         wait for 200 ns;
58
59         -- 仿真结束
60         wait;
61     end process;
62
63 end tb;

```

1.3 结果推导

本序列发生器基于 Moore 型有限状态机 (FSM) 原理进行设计，其核心机制依赖于同步时序状态跳转与组合逻辑输出的配合。当外部复位信号 reset 为高电平有效时，系统被强行初始化至起始状态 000；复位释放后，在每一个时钟信号 clk 的上升沿驱动下，状态机严格按照预设的状态转移逻辑，依序在 000、001、010、011、100、101、110 之间进行循环跳转。由于系统采用 Moore 机架构，其输出 dataout 仅严格取决于当前所处的状态节点，组合逻辑电路对这 7 个连续状态进行解码，分别对应映射出 1、1、0、1、1、1、0 的电平值，从而在连续的时钟节拍推动下，稳定且循环地生成目标序列 “1101110”。

• 序列信号发生器

按需要产生一组特定串行序列的电路被称为序列信号发生器。下图为 “1101110” 序列生成器的模块框图和状态转移图。如图所示，发生器有2个输入：第一个是复位信号，用于将状态初始化为000；第二个是时钟输入。

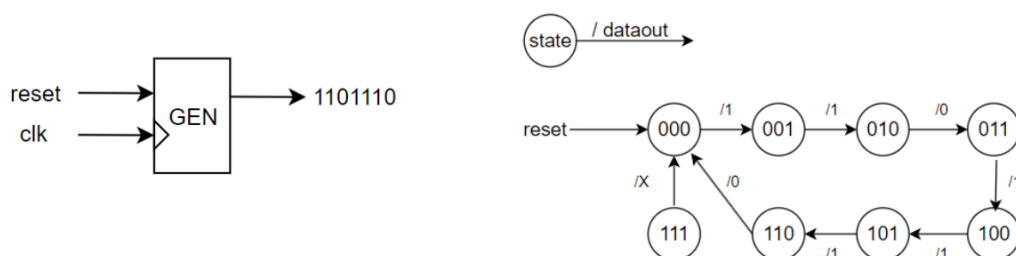


图 1: principle

1.4 结果

1.4.1 behavior simulation

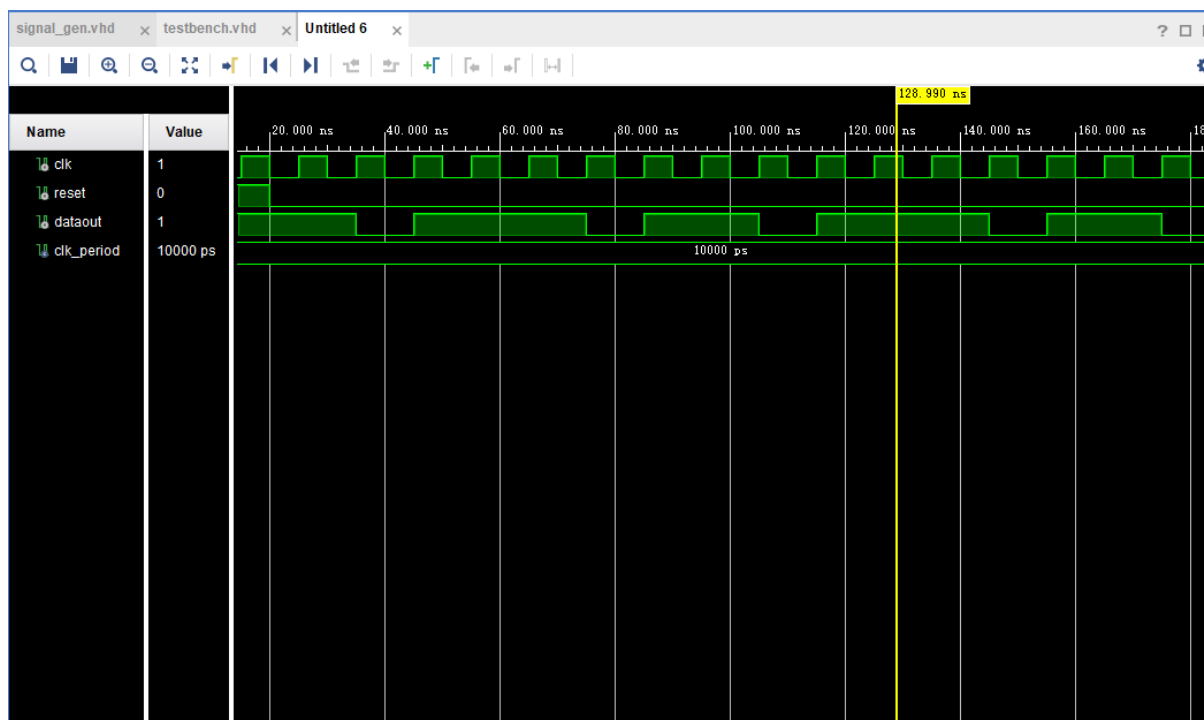


图 2: Behavior Simulation Result

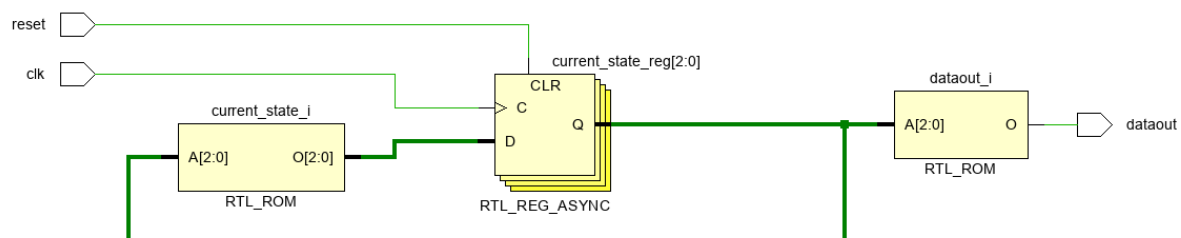


图 3: Behavior Schematic Simulation Result

1.4.2 post synthesis timing simulation

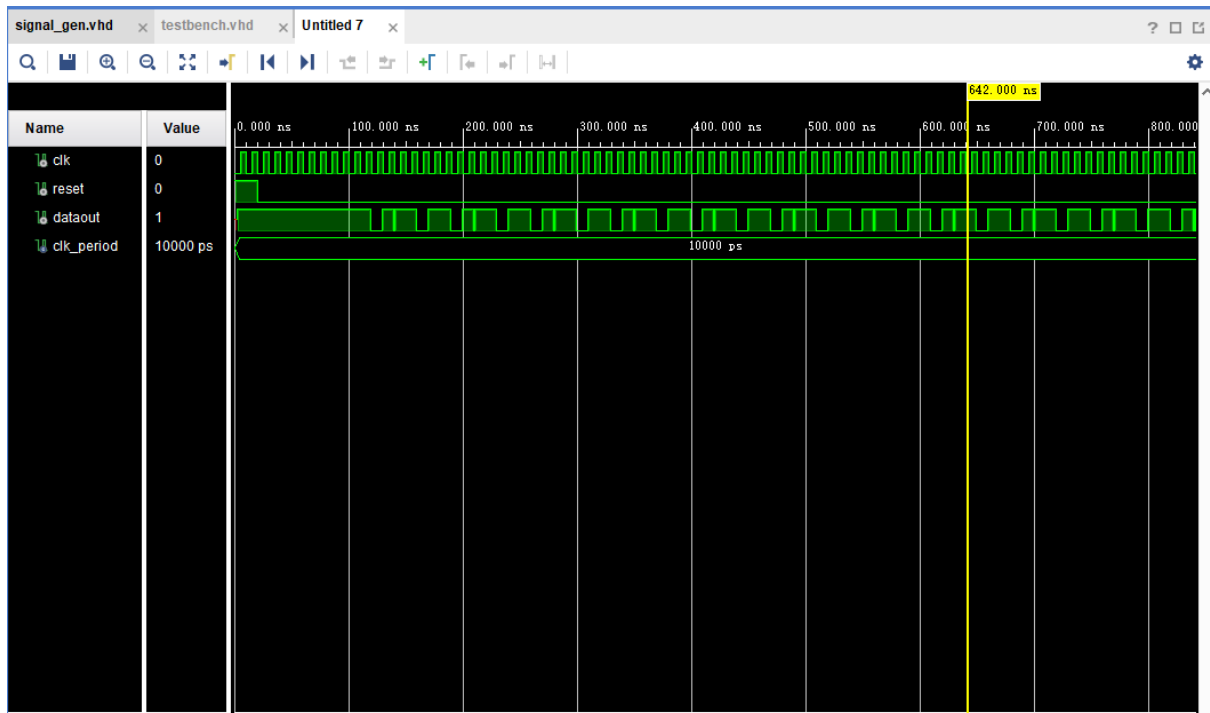


图 4: Post Synthesis Timing Simulation Result

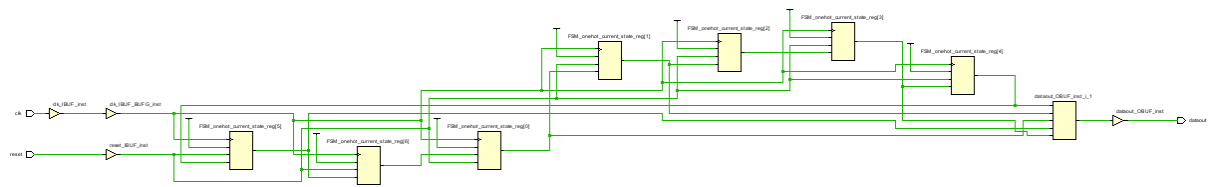


图 5: Synthesis Schematic Simulation Result

1.4.3 post implementation timing simulation

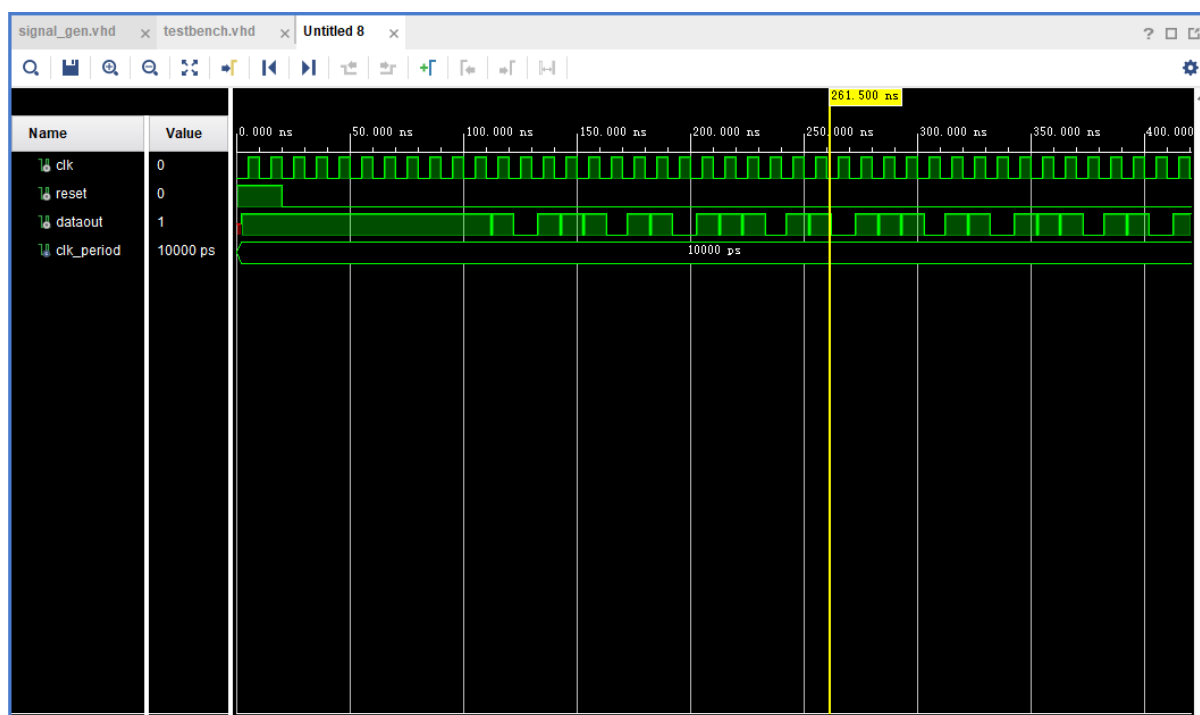


图 6: Post Implementation Timing Simulation Result

可以看出，随着时钟以及 reset 触发，系统持续输出 ‘1101110’ 并进行循环。

1.5 分析

通过对比可知，行为级仿真（Behavioral Simulation）呈现的是一种理想化的时序关系，其波形与 VHDL 源码的逻辑预期高度吻合。由于该阶段未计入任何物理延迟，信号的跳变瞬间完成，其核心目的在于纯粹的功能性验证，即确认状态机的逻辑转移是否无误。

相比之下，综合后时序仿真（Post-Synthesis Simulation）引入了逻辑门级的固有延迟。由于电路的状态转换依赖于底层的组合逻辑网表，门电路传输时间的不一致导致了信号到达的先后差异，进而在波形中引发了竞争冒险（毛刺）现象。这直观反映了物理器件本身的开关特性对电路实际运行情况的制约。

实现后时序仿真（Post-Implementation Simulation）则是最逼近芯片真实物理工作环境的验证环节。该阶段不仅涵盖了器件延迟，更全面叠加了布局布线后产生的复杂路径延迟（Routing Delay）。这使得最终的输出波形与理想的逻辑代码预期产生了显著的时序偏移与形态差异，从而高保真地还原了硬件电路的实际物理行为。

1.6 结论

本次实验成功完成了 “1101110” 序列发生器的 VHDL 设计与 Testbench 验证。通过递进对比行为级、综合后及实现后三种仿真，实验不仅确认了 FSM 状态转移逻辑的正确性，更直观观察到了由底层器件延迟和物理布线延迟所引发的时序偏移与竞争冒险现象。这加深了对理想逻辑与真实硬件差异的理解，完整掌握了从代码编写到后端时序分析的 EDA 开发流程。

2 lab4.3 序列信号检测器

2.1 实验目的

本实验旨在通过 VHDL 硬件描述语言设计并实现一个具有状态锁定与复位功能的“101”序列检测器，从而深入理解有限状态机（FSM）在数字逻辑中的应用，并熟练运用 Testbench 进行综合及后仿真（Post-implementation simulation），以验证电路设计的正确性与时序特征。

2.2 VHDL 代码和 testbench 代码

本部分为 101 序列检测器的核心硬件描述代码。代码采用经典的“两段式”有限状态机（FSM）结构编写：第一个进程（时序逻辑）负责处理基于时钟沿的同步状态更新与最高优先级的异步复位操作；第二个进程（组合逻辑）则严格根据状态转移图，结合当前状态和输入数据（datain）推导次态，并精准控制检测到“101”后的输出跳变与状态锁定功能。

design 代码如下：

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity seq_detect is
5      Port ( clk : in STD_LOGIC;
6            reset : in STD_LOGIC;
7            datain : in STD_LOGIC;
8            match : out STD_LOGIC);
9  end seq_detect;
10
11 architecture Behavioral of seq_detect is
12     -- 定义状态机的四个状态
13     type state_type is (S00, S01, S10, S11);
14     signal current_state, next_state : state_type;
15 begin
16
17     -- 进程1：时序逻辑，负责状态更新（包含异步复位）
18     process(clk, reset)
19     begin
20         if reset = '1' then
21             current_state <= S00;
22         elsif rising_edge(clk) then
23             current_state <= next_state;
24         end if;
25     end process;
26
27     -- 进程2：组合逻辑，负责状态转移条件和输出
28     process(current_state, datain)
29     begin
30         -- 默认输出为0，避免产生锁存器(Latch)
31         match <= '0';
32
33         case current_state is
34             when S00 =>
35                 if datain = '1' then
36                     next_state <= S01;
37                 else
38                     next_state <= S00;
39                 end if;
40             when S01 =>
```



```

41         when S01 =>
42             if datain = '0' then
43                 next_state <= S10;
44             else
45                 next_state <= S01;
46             end if;
47
48         when S10 =>
49             if datain = '1' then
50                 next_state <= S11;
51                 match <= '1'; -- 转移到S11时输出1（对应图中的 1/1）
52             else
53                 next_state <= S00;
54             end if;
55
56         when S11 =>
57             next_state <= S11; -- 锁定状态
58             match <= '1';      -- 保持输出为1（对应图中的 0/1, 1/1）
59
60         when others =>
61             next_state <= S00;
62     end case;
63 end process;
64
65 end Behavioral;

```

本部分为验证序列检测器逻辑功能的仿真测试平台代码。该模块不含实际输入输出端口，主要用于实例化待测设计（UT）并为其提供测试激励。代码内部模拟生成了固定周期的系统时钟信号（clk），并精心构造了“初始复位——输入包含‘101’的特定数据流——验证输出锁定——再次复位——复位后再次验证功能完整性”的完整测试场景，旨在通过波形仿真全面验证设计是否满足题目的各项功能要求。

testbench 代码如下：

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity testbench is
5      -- Testbench 没有端口
6  end testbench;
7
8  architecture tb of testbench is
9      -- 声明待测试组件
10     component seq_detect
11     port(
12         clk : in std_logic;
13         reset : in std_logic;
14         datain : in std_logic;
15         match : out std_logic
16     );
17     end component;
18
19     -- 内部信号定义
20     signal clk : std_logic := '0';
21     signal reset : std_logic := '0';
22     signal datain : std_logic := '0';
23     signal match : std_logic;
24

```

```

25     -- 定义时钟周期
26     constant clk_period : time := 10 ns;
27
28 begin
29     -- 实例化被测设计 (UUT)
30     uut: seq_detect port map (
31         clk => clk,
32         reset => reset,
33         datain => datain,
34         match => match
35     );
36
37     -- 时钟生成进程
38     clk_process : process
39     begin
40         clk <= '0';
41         wait for clk_period/2;
42         clk <= '1';
43         wait for clk_period/2;
44     end process;
45
46     -- 激励信号产生进程
47     stim_proc: process
48     begin
49         -- 1. 系统复位
50         reset <= '1';
51         wait for 20 ns;
52         reset <= '0';
53         wait for 10 ns;
54
55         -- 2. 开始输入序列测试
56         -- 尝试输入: 0 -> 1 -> 1 -> 0 -> 1 (此处应触发match) -> 0 -> 0
57         datain <= '0'; wait for clk_period;
58         datain <= '1'; wait for clk_period; -- 状态 -> S01
59         datain <= '1'; wait for clk_period; -- 状态保持 S01
60         datain <= '0'; wait for clk_period; -- 状态 -> S10
61
62         datain <= '1'; wait for clk_period; -- 状态 -> S11, 此时 match 变为 1
63
64         -- 3. 验证锁定功能 (继续输入数据, 看 match 是否保持为 1)
65         datain <= '0'; wait for clk_period;
66         datain <= '0'; wait for clk_period;
67
68         -- 4. 验证复位功能
69         reset <= '1'; wait for clk_period;
70         reset <= '0'; wait for clk_period;
71
72         -- 5. 验证正常功能
73         datain <= '1'; wait for clk_period * 2;
74         datain <= '0'; wait for clk_period;
75         datain <= '1'; wait for clk_period;
76
77
78         wait; -- 停止仿真
79     end process;
80
81 end tb;

```

2.3 结果推导

本设计采用 Mealy 型有限状态机 (FSM) 实现“101”序列检测，通过定义 S0 至 S3 四个逻辑状态分别记录匹配进度：系统从初始态出发，经由有效位依次跳转，直至在 S2 状态接收到关键输入 1 时判定匹配成功；为满足输出锁定需求，将 S3 设为吸收态 (Absorbing State)，使其在触发后无视后续输入并持续保持高电平输出，仅在全局 reset 信号激活时强制回归初始态，从而实现逻辑闭环。

• 101序列检测器

以下为序列检测器的模块框图和状态转移图。如图所示，检测器有3个输入：

第一个是复位信号，用于将状态初始化为00；

第二个是位串行输入；

第三个是时钟输入。

检测器能够识别输入序列中的 101 序列，并将输出值设置为 1。

输出值将始终保持为 1，直到被复位。

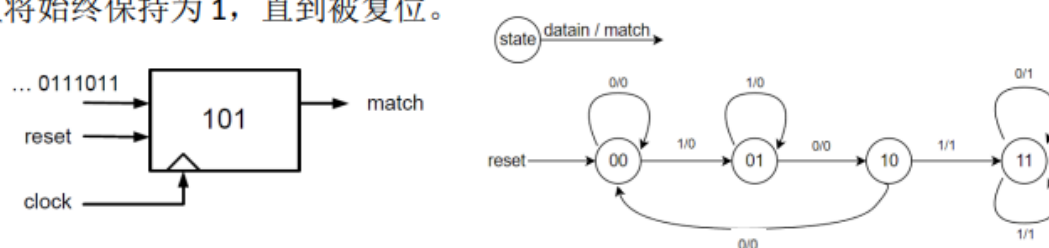


图 7: principle

2.4 结果

2.4.1 behavior simulation

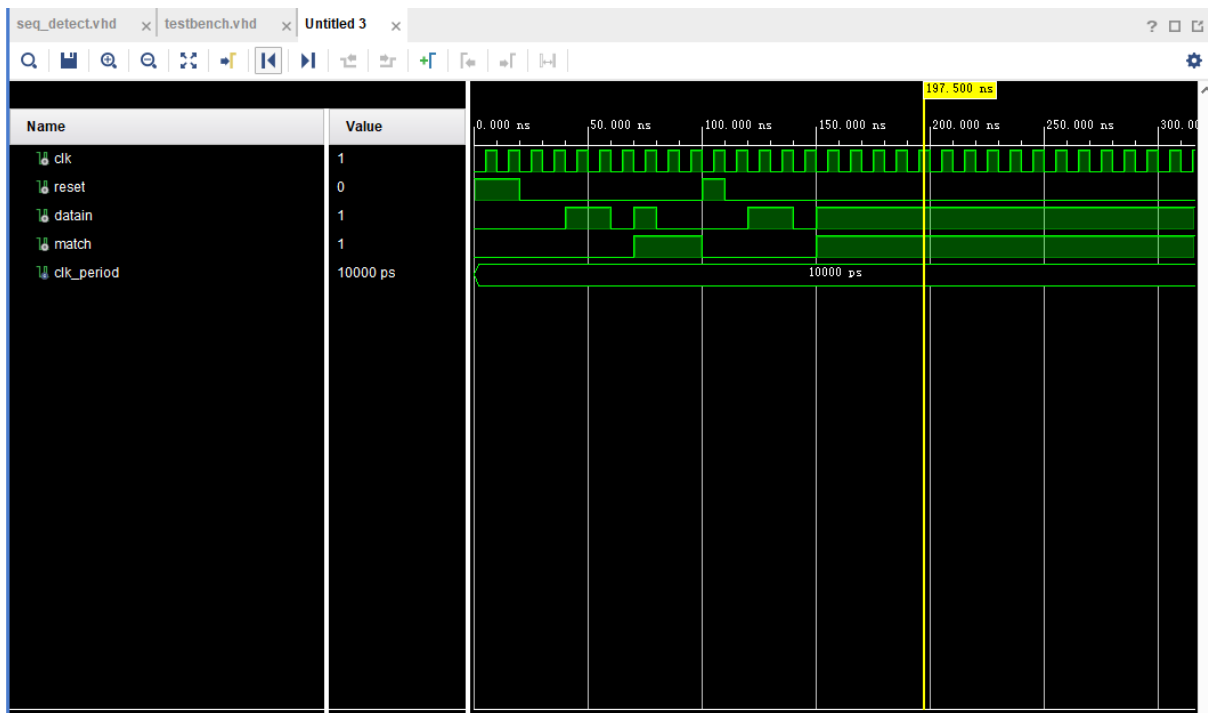


图 8: Behavior Simulation Result

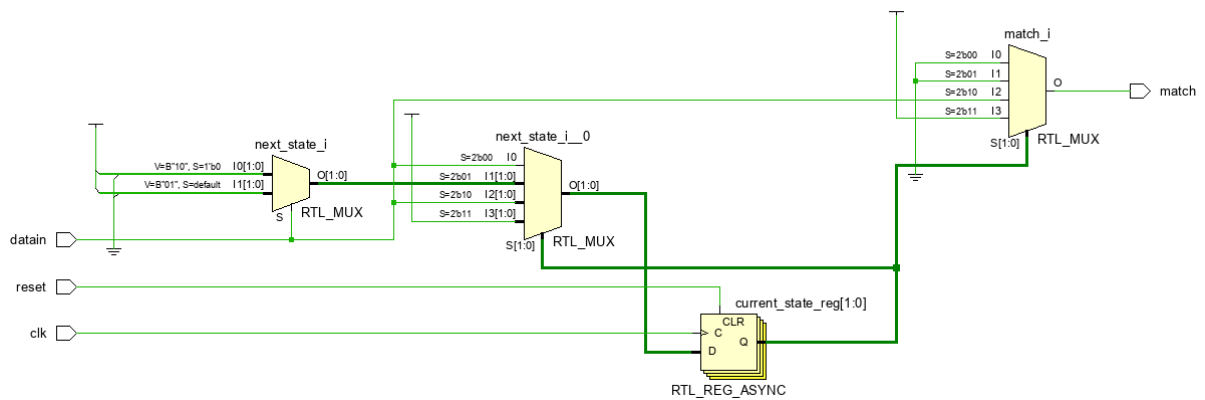


图 9: Behavior Schematic Simulation Result

2.4.2 post synthesis timing simulation

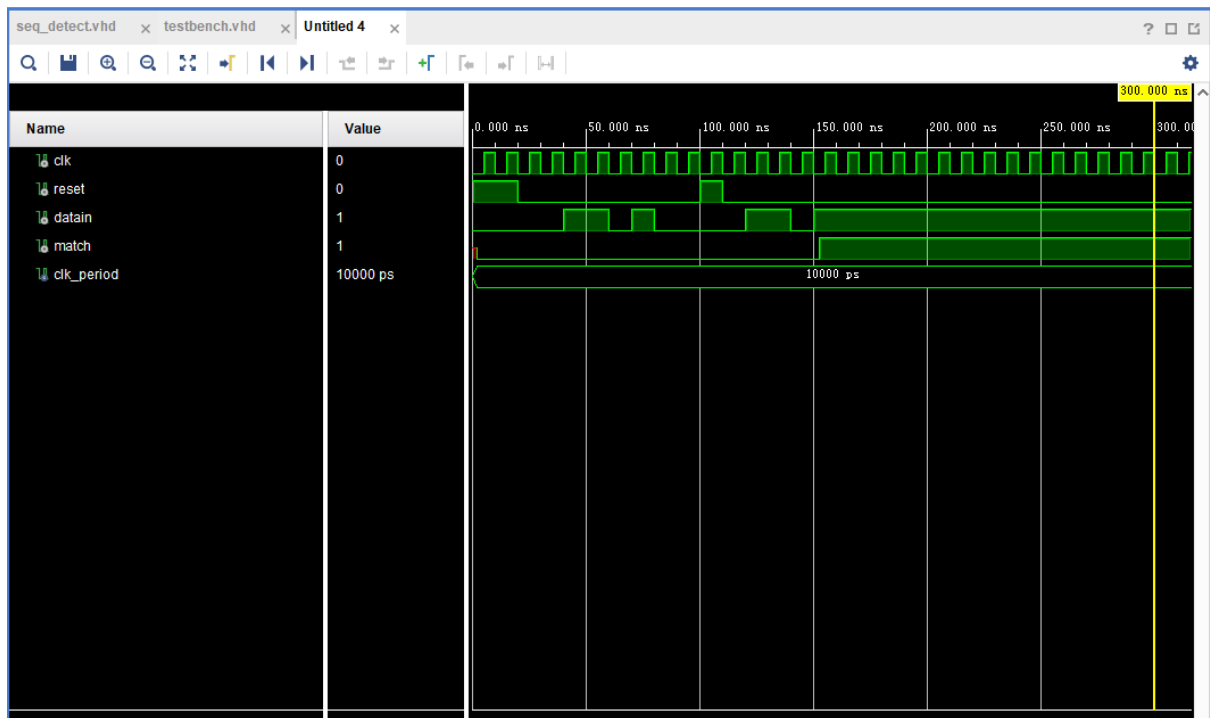


图 10: Post Synthesis Timing Simulation Result

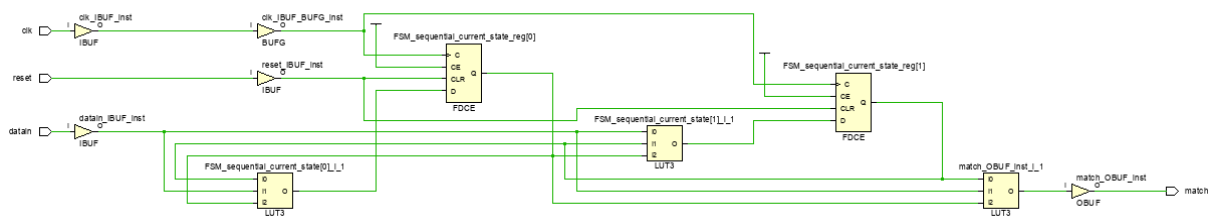


图 11: Synthesis Schematic Simulation Result

2.4.3 post implementation timing simulation

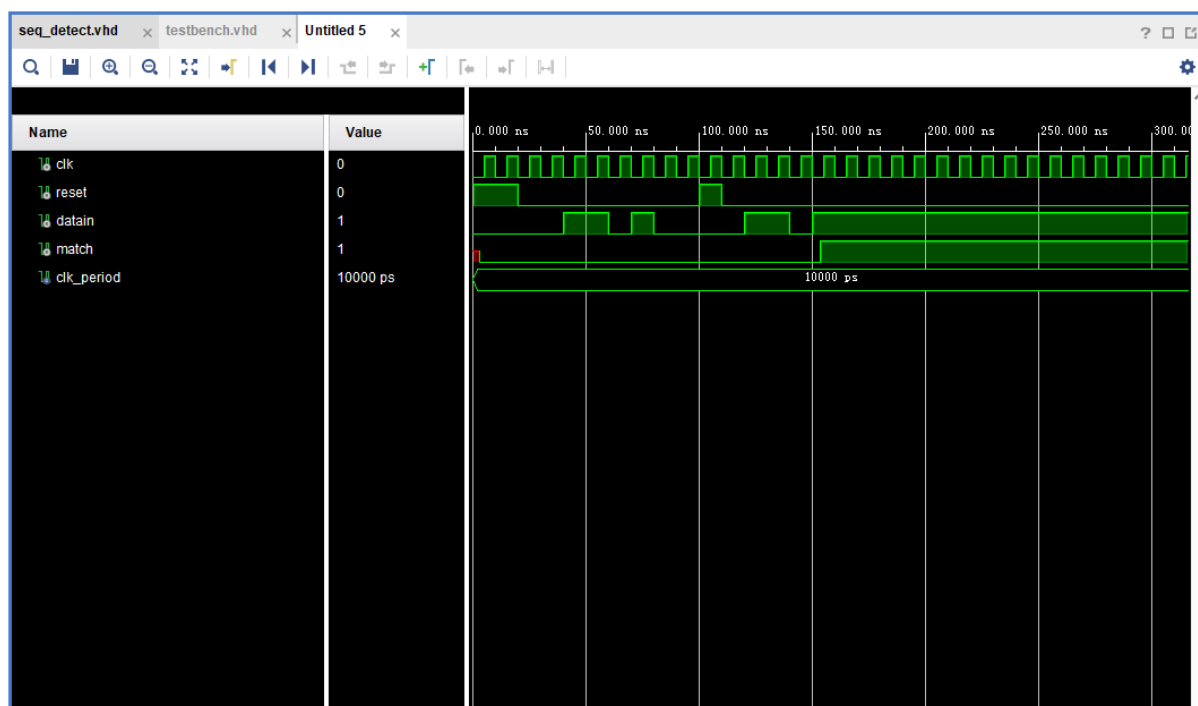


图 12: Post Implementation Timing Simulation Result

从图中可以看出，随着时钟与 reset 的触发，先输入 ‘101’，match 可以正常输出 ‘1’，再输入 ‘0’，验证 match 可以锁定输出 ‘1’；再进行复位，确定复位后，match 归零；最后输入 ‘101’ 确保复位后功能保持正常，match 输出 ‘1’。

2.5 分析

对比功能仿真，后时序仿真揭示了硬件实现中的物理特性影响。受限于器件固有延迟，match 信号的响应滞后于输入激励，甚至在特定的 101 信号片段中，因时序裕量耗尽而出现检测漏报。此外，由于信号到达逻辑门的时间不一致，波形在状态切换瞬间产生了非预期的高频振荡毛刺，反映了实际电路中的异步时序风险。

2.6 结论

通过对 Mealy 状态机的仿真对比，我们观察到硬件物理特性对逻辑行为的深刻影响。实验揭示了门级延迟如何将逻辑上的“完美序列”转化为时域上的“非预期脉冲（毛刺）”，甚至导致状态转移失效。这一结果强调了数字逻辑设计不能脱离物理载体，必须在设计初期便充分考虑时序裕量 (Timing Margin)，通过优化逻辑深度与布局布线，提升电路在真实动态环境下的抗干扰能力与可靠性。